Database Management Systems
CPS 3740

Lecture 12

Computer Science, Kean University
Dr. Ching-yu (Austin) Huang
Instructor Paolien Wang

Outline

- Chapter 4
  - Relational model
- Chapter 5
  - Relational algebra
  - Relation calculus (optional)

Chapter 4

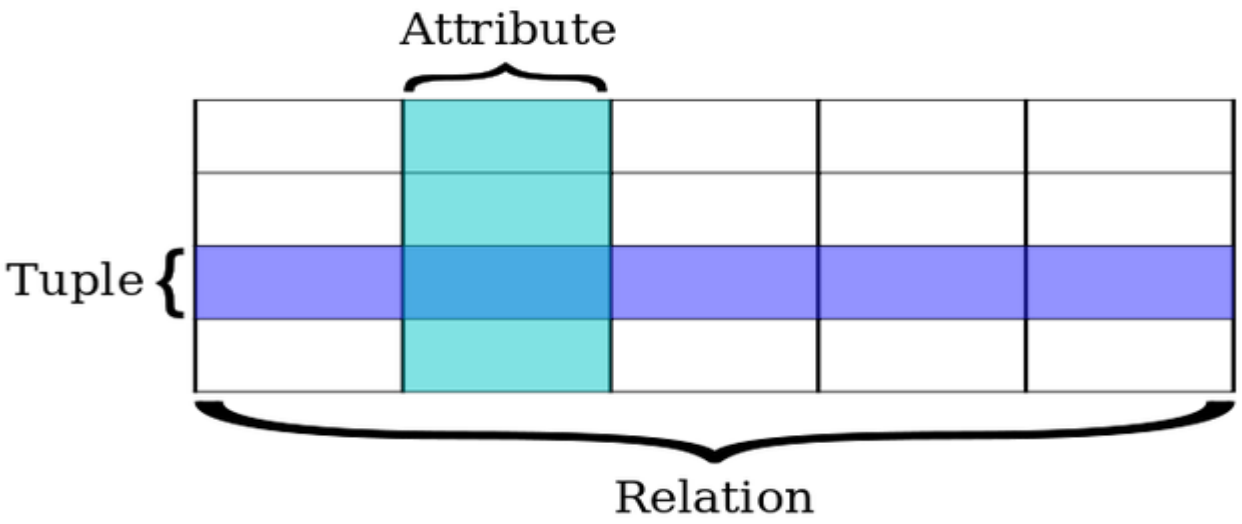
The Relational Model

Relational Model Terminology

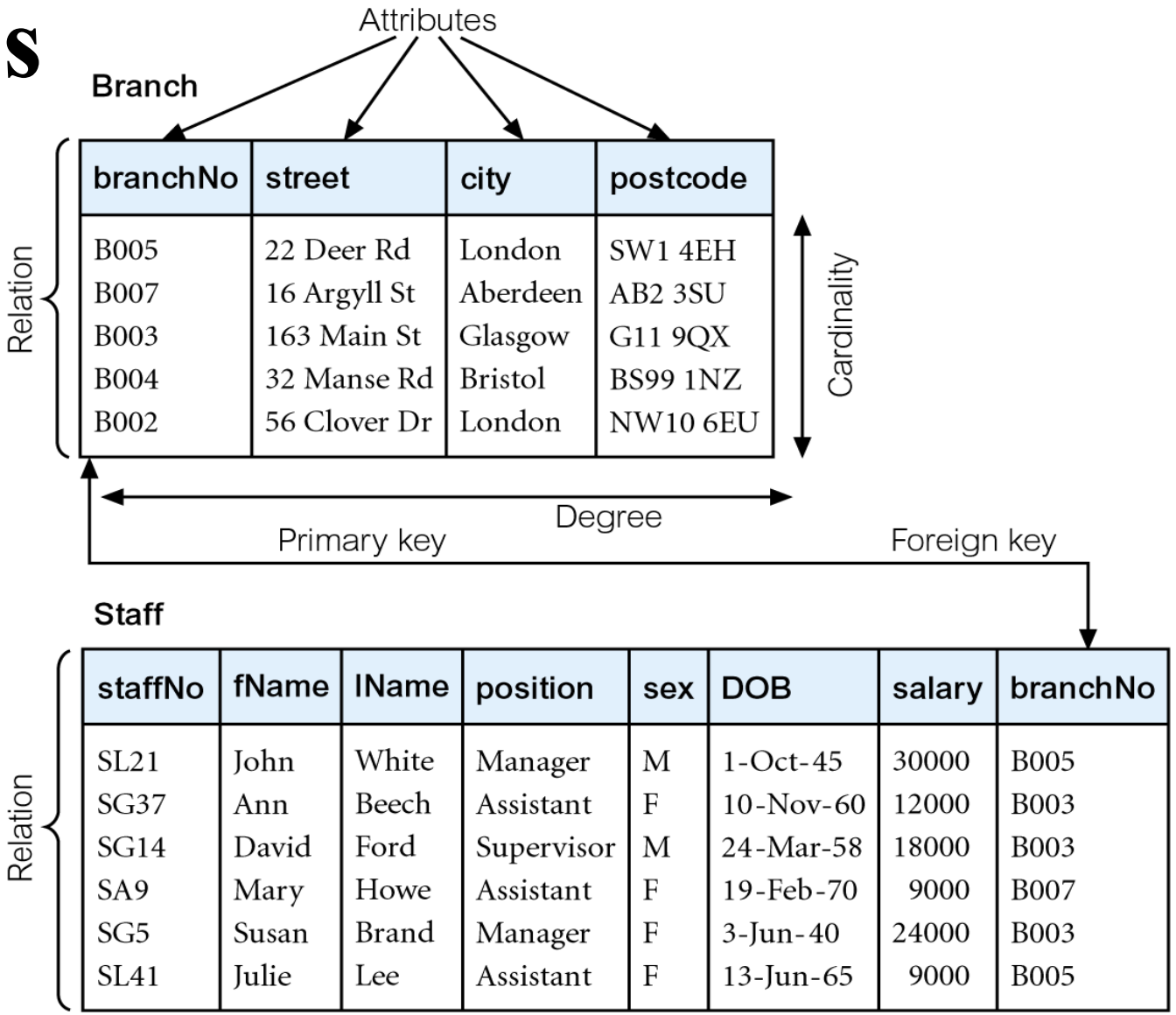
- A relation is a table with columns and rows.
  - Only applies to logical structure of the database, not the physical structure.
- Attribute is a named column of a relation.
- Domain is the set of allowable values for one or more attributes.

Relational Model Terminology

- Tuple is a row of a relation.
- Degree is the number of attributes in a relation.
- Cardinality is the number of tuples in a relation.
- Relational Database is a collection of normalized relations with distinct relation names.



Instances of Branch and Staff Relations



Branch

branchNo	street	city	postcode
B005	22 Deer Rd	London	SW1 4EH
B007	16 Argyll St	Aberdeen	AB2 3SU
B003	163 Main St	Glasgow	G11 9QX
B004	32 Manse Rd	Bristol	BS99 1NZ
B002	56 Clover Dr	London	NW10 6EU

Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

Guideline for Relation Design

- Avoid duplicate columns in different tables
  - Possible cause inconsistency
- Will discuss data anomalies and database normalization later

Customers			
C_ID	C_Name	Address	gender

Orders					
O_ID	C_Name	Address	P_Name	Quantity	Price

Mathematical Definition of Relation

- Consider two sets, D_1 & D_2, where $D_1 = \{2, 4\}$ and $D_2 = \{1, 3, 5\}$.

  - **Cartesian product**, $D_1 \times D_2$, is set of all ordered pairs, where first element is member of D_1 and second element is member of D_2.

$$D_1 \times D_2 = \{(2, 1), (2, 3), (2, 5), (4, 1), (4, 3), (4, 5)\}$$

  - All combinations of (D_1 , D_2)

Mathematical Definition of Relation

- Consider two sets, D_1 & D_2, where $D_1 = \{2, 4\}$ and $D_2 = \{1, 3, 5\}$.

  - **Any subset of Cartesian product is a relation**; e.g.

    $R = \{(2, 1), (4, 1)\}$

  - May specify which pairs are in relation using some condition for selection;

    e.g.

    - second element is 1:

      $R = \{(x, y) \mid x \in D_1, y \in D_2, \text{ and } y = 1\}$

Example - Cartesian product

student_academic(slno, class, total)

slno	class	total
208501	S5R	211
208502	S5R	137
208503	S5R	210

student_personal(sno, sname, city)

sno	sname	city
208501	ANIL	EKM
208502	BABU	TVM

3 x 2

➔ 6 rows

With all columns

student_academic X student_personal

slno	class	total	sno	sname	city
208501	S5R	211	208501	ANIL	EKM
208502	S5R	137	208502	BABU	TVM
208503	S5R	210	208501	ANIL	EKM
208501	S5R	211	208502	BABU	TVM
208502	S5R	137	208501	ANIL	EKM
208503	S5R	210	008502	BABU	TVM

Database Relations

- Relation schema
  - Named relation defined by a set of attribute and domain name pairs.
- Relational database schema
  - Set of relation schemas, each with a distinct name.
  - Schema example:

    Staff(staffno, name, sex, position, branchno)

Properties of Relations

- Relation name is distinct from all other relation names in relational schema.
- Each cell of relation contains exactly one atomic (single) value.
- Each attribute has a distinct name.
- Values of an attribute are all from the same domain. (same data type)

Properties of Relations

- Each tuple is distinct; there are no duplicate tuples.
- Order of attributes has no significance.
- Order of tuples has no significance, theoretically.

Relational Keys

- Superkey
  - An attribute, or set of attributes, that uniquely identifies a tuple within a relation.

  (SID, SSN, Birthday, gender, name, address, DNA)
- Candidate Key (minimal SuperKey)
  - Superkey (K) such that no proper subset is a superkey within the relation.
  - In each tuple of R, values of K uniquely identify that tuple (uniqueness).
  - No proper subset of K has the uniqueness property (irreducibility).

Candidate Keys

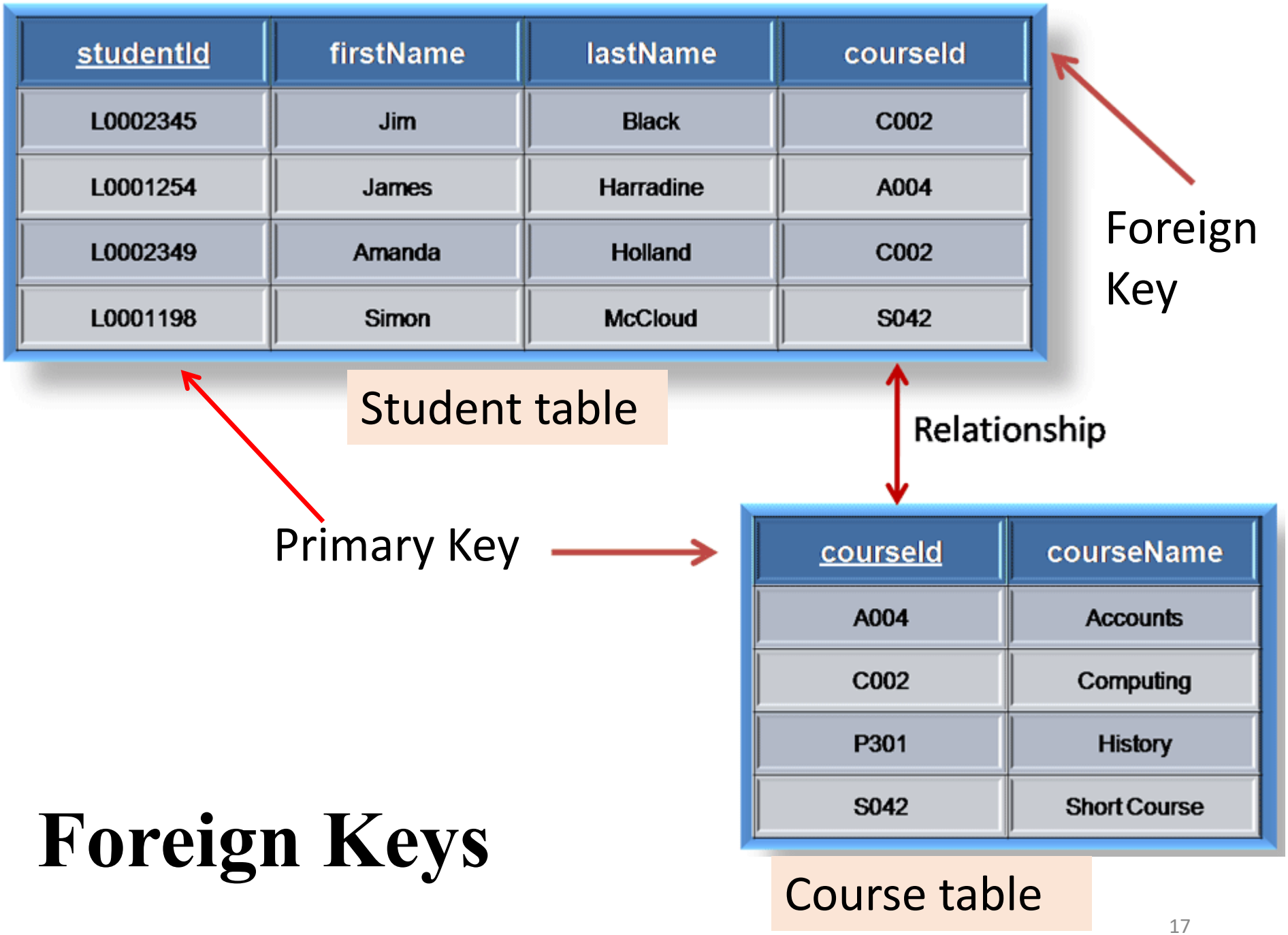
StudentId	firstName	lastName	courseId
L0002345	Jim	Black	C002
L0001254	James	Harradine	A004
L0002349	Amanda	Holland	C002
L0001198	Simon	McCloud	S042
L0023487	Peter	Murray	P301
L0018453	Anne	Norris	S042

Student table: Assume every student has different firstName + lastName

Relational Keys

- Primary Key
  - Candidate key selected to identify tuples uniquely within relation.
  - Example: StudentId in the Student Table
- Alternate Keys
  - Candidate keys that are not selected to be primary key. Example: firstName+lastName in the Student Table.
- Foreign Key
  - Attribute, or set of attributes, within one relation that matches candidate key of some (possibly same) relation.
  - Example: courseId in the Student Table
  - Values in the Foreign Key are not necessary unique.

Foreign Keys



How About These Tables?

Suit	Value	No. times played
Hearts	Ace	5
Diamonds	Three	2
Hearts	Jack	3
Clubs	Three	5
Spades	Five	1

One is not enough to identify, but both combined make a unique value

Composite Keys

- A **simple key** is one that has only one attribute.
- A composite key contains two or more attributes.
- A primary key can be a composite key.

Buildings					
B_ID	B_Name	Room#	Location	Type	Capacity

Courses						
Year	Term	Major	Course#	Section	Instructor	B_ID

The Keys in Database

Integrity Constraints

- Null
  - Represents value for an attribute that is currently unknown or not applicable for tuple.
  - Deals with incomplete or exceptional data.
  - Represents the absence of a value and is not the same as zero or spaces, which are values.
  - NULL is “unknown”, NOT empty, NOT “”, NOT 0

Null Operations

p	*q*	*p* OR *q*	*p* AND *q*	*p* = *q*
True	True	True	True	True
True	False	True	False	False
True	Unknown	True	Unknown	Unknown
False	True	True	False	False
False	False	False	False	True
False	Unknown	Unknown	False	Unknown
Unknown	True	True	Unknown	Unknown
Unknown	False	Unknown	False	Unknown
Unknown	Unknown	Unknown	Unknown	Unknown

p	NOT *p*
True	False
False	True
Unknown	Unknown

Chapter 5

Relational Algebra and Relational Calculus

Relational Algebra

- Relational algebra and relational calculus are formal languages associated with the relational model.
- Relational algebra operations work on one or more relations to define another relation without changing the original relations.

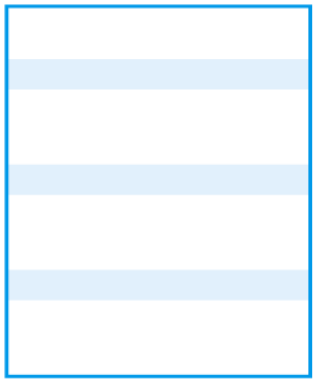
Relational Algebra

- Five basic operations in relational algebra: Selection, Projection, Cartesian product, Union, and Set Difference.

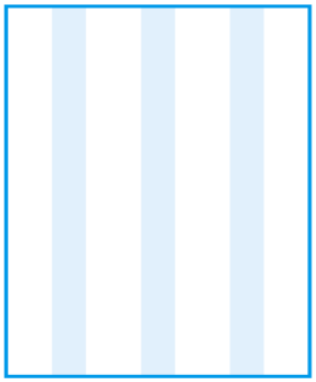
- These perform most of the data retrieval operations needed.

- Also have Join, Intersection, and Division operations, which can be expressed in terms of 5 basic operations.

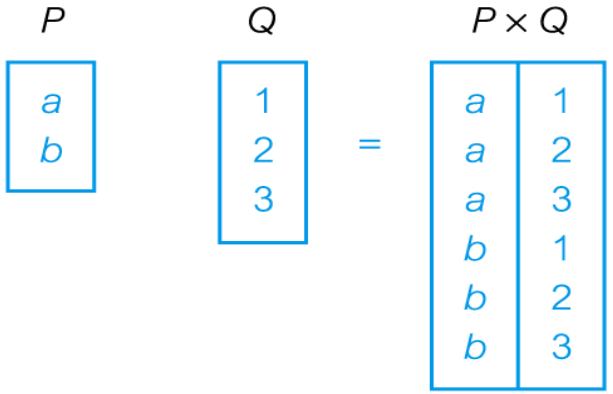
Relational Algebra Operations



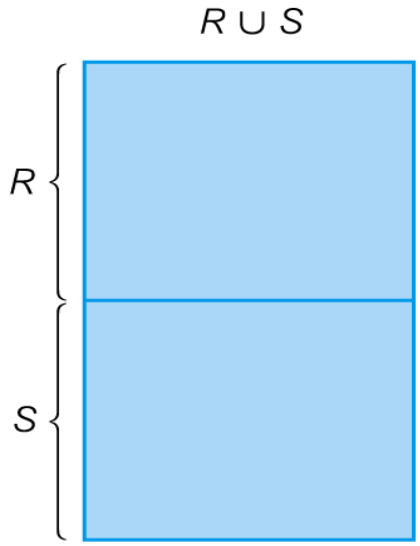
(a) Selection



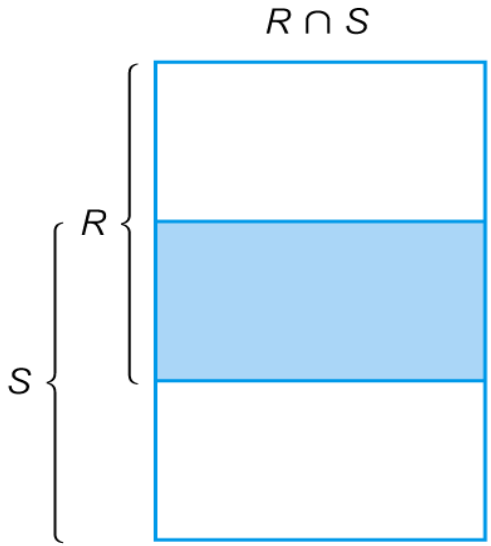
(b) Projection



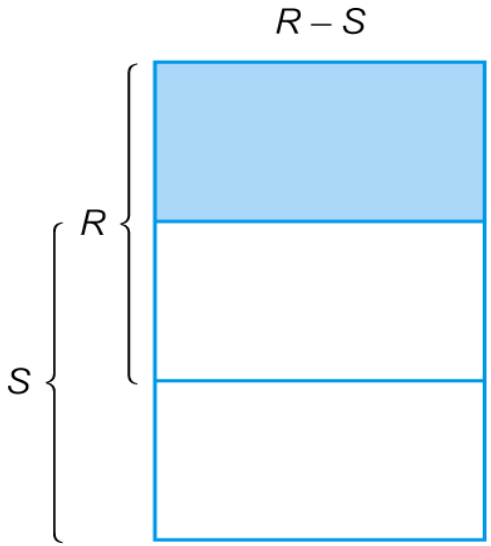
(c) Cartesian product



(d) Union



(e) Intersection



(f) Set difference

Selection (or Restriction)

- All operation symbols are listed in Table 5.1 on page 131
- $\sigma_{\text{predicate}}$ **(R)**
  - Works on a single relation (table) **R** and defines a relation that contains only those tuples (rows) of R that satisfy the specified condition (*predicate*).

Example - Selection (or Restriction)

- List all staff with a salary greater than 10,000.

$\sigma_{salary > 10000}$ **(Staff)**

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24- Mar-58	18000	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003

Projection

- $\Pi_{col1, \ldots, coln}(R)$
  - Works on a single relation R and defines a relation that contains a vertical subset of R, extracting the values of specified attributes and eliminating duplicates.

Example - Projection

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

- Produce a list of salaries for all staff, showing only staffNo, fName, lName, and salary details.

$\Pi_{\text{staffNo, fName, lName, salary}}(\text{Staff})$

staffNo	fName	lName	salary
SL21	John	White	30000
SG37	Ann	Beech	12000
SG14	David	Ford	18000
SA9	Mary	Howe	9000
SG5	Susan	Brand	24000
SL41	Julie	Lee	9000

Example – Selection, Projection

$\sigma_{\text{salary >= 50000}}$ **(Employee)**

Employee				
E_ID	Name	Dept	Salary	L_ID
1	Austin	IT	50000	1
3	Sam	IT	53000	2

$\Pi_{\text{Name, Dept, Salary}}$**(Employee)**

Employee		
Name	Dept	Salary
Austin	IT	50000
Mary	HR	45000
Sam	IT	53000
Andrew	Sales	42000

Employee				
E_ID	Name	Dept	Salary	L_ID
1	Austin	IT	50000	1
2	Mary	HR	45000	3
3	Sam	IT	53000	2
4	Andrew	Sales	42000	1

Example – Selection & Projection

$$\Pi_{\text{Name, Dept, Salary}}(\sigma_{\text{salary >= 50000}}(\text{Employee}))$$

```
Select Name, Dept, Salary
From Employee
Where salary >= 50000
```

Employee		
Name	Dept	Salary
Austin	IT	50000
Sam	IT	53000

Employee				
E_ID	Name	Dept	Salary	L_ID
1	Austin	IT	50000	1
2	Mary	HR	45000	3
3	Sam	IT	53000	2
4	Andrew	Sales	42000	1

Union

- **R ∪ S**
  - Union of two relations R and S defines a relation that contains all the tuples of R, or S, or both R and S, duplicate tuples being eliminated.
  - R and S must be union-compatible → the relations must have the same number of attributes, but not necessary same attribute names.

- If R and S have *I* and *J* tuples, respectively, union is obtained by concatenating them into one relation with a maximum of (*I* + *J*) tuples.

Example - Union

- List all cities where there is either a branch office or a property for rent. (Fig 4-3, page 112)

$$\Pi_{city}(\textbf{Branch}) \cup \Pi_{city}(\textbf{PropertyForRent})$$

(select city from Branch) UNION(select city from PropertyForRent);

city
London
Aberdeen
Glasgow
Bristol
London

∪

city
Aberdeen
London
Glasgow
Glasgow
Glasgow
Glasgow

→

city
London
Aberdeen
Glasgow
Bristol

Set Difference

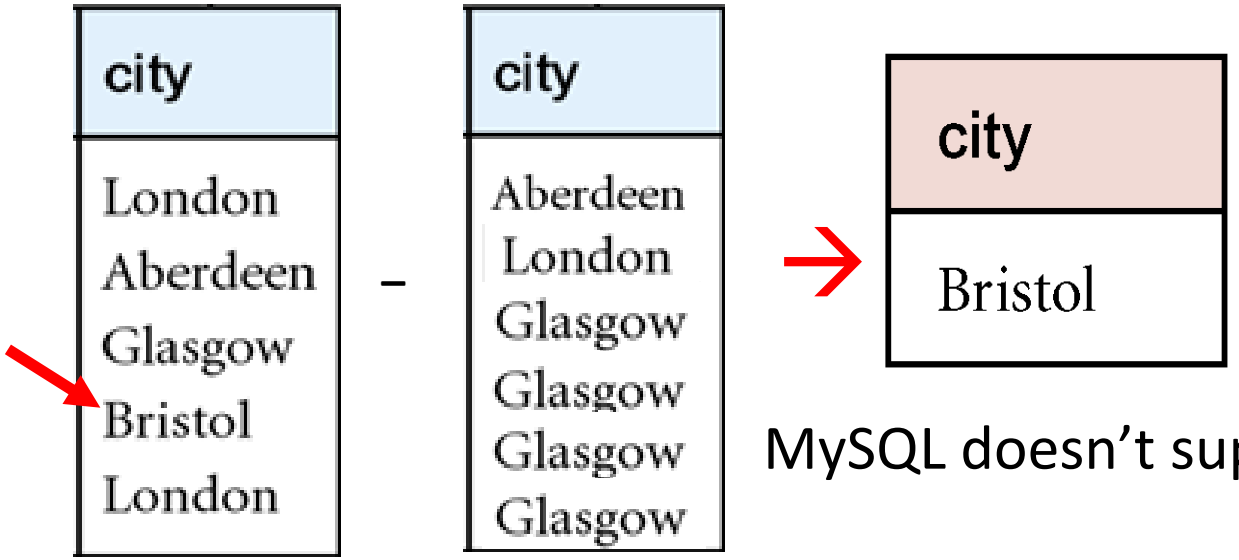
- **R – S**
  - Defines a relation consisting of the tuples that are in relation R, but not in S.
  - R and S must be union-compatible.

Example - Set Difference

- List all cities where there is a branch office but no properties for rent.

$$\Pi_{city}(\textbf{Branch}) - \Pi_{city}(\textbf{PropertyForRent})$$

```
select city from Branch where
 city not in (select city from PropertyForRent);
```



MySQL doesn't support MINUS, EXCEPT

Intersection

- **$R \cap S$**
  - Defines a relation consisting of the set of all tuples that are in both R and S.
  - R and S must be union-compatible.

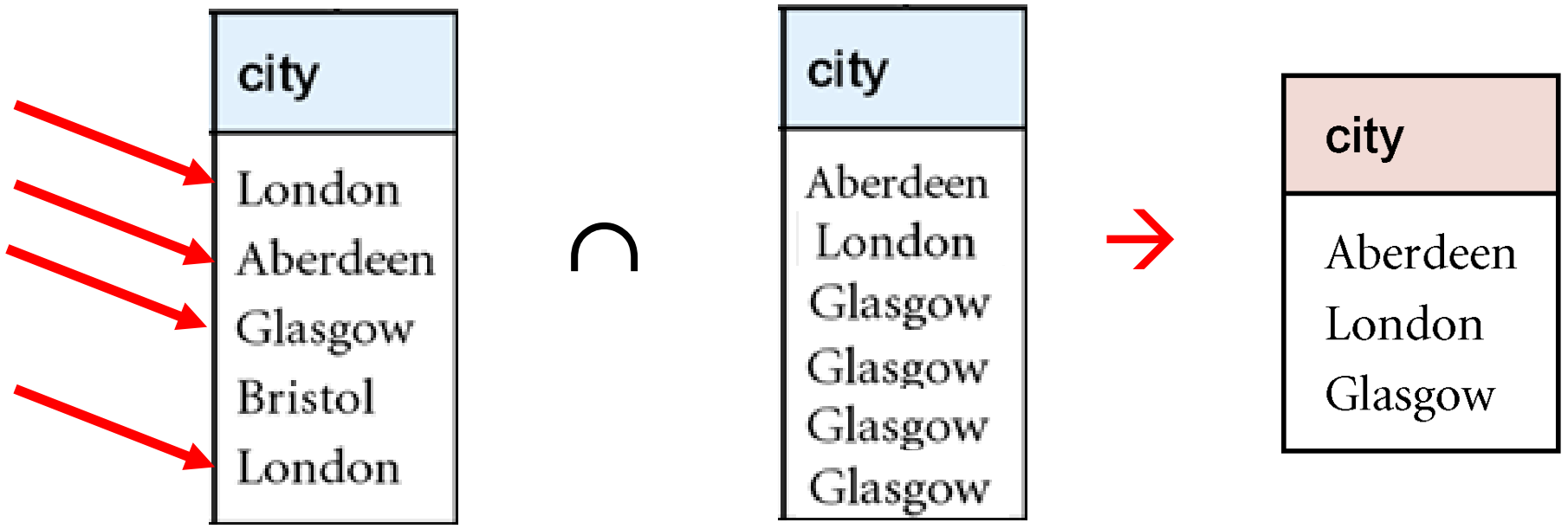
- Expressed using basic operations:

  $R \cap S = R - (R - S)$

Example - Intersection

- List all cities where there is both a branch office and at least one property for rent.

$$\Pi_{city}(\textbf{Branch}) \cap \Pi_{city}(\textbf{PropertyForRent})$$



Aggregate Operations

- $\mathfrak{I}_{AL}(R)$
  - Applies aggregate function list, **AL**, to R to define a relation over the aggregate list.
  - AL contains one or more (<aggregate_function>, <attribute>) pairs .
- Main aggregate functions are: COUNT, SUM, AVG, MIN, and MAX.

Example – Aggregate Operations

- ρ → rename a column
- How many properties cost more than 350 per month to rent?

myCount
5

$\rho_R(\text{myCount})\ \Im_{\text{COUNT propertyNo}}\ (\sigma_{\text{rent} > 350}\ (\text{PropertyForRent}))$

select Count(propertyNo) as myCount from PropertyForRent where rent > 350;

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650.00	CO46	SA9	B007
PG16	5 Novar Drive	Glasgow	G12 9AX	Flat	4	450.00	CO93	SG14	B003
PG21	18 Dale Road	Glasgow	G12	House	5	600.00	CO87	SG37	B003
PG36	2 Manor Road	Glasgow	G32 4QX	Flat	3	375.00	CO93	SG37	B003
PG4	6 Lawrence Street	Glasgow	G11 9QX	Flat	3	350.00	CO40	NULL	B003
PL94	6 Argyll Street	London	NW2	Flat	4	400.00	CO87	SL41	B005
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Grouping Operation

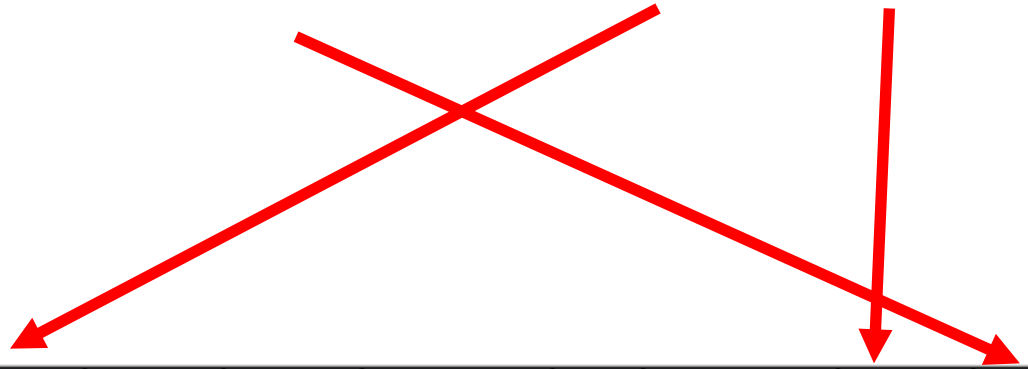
- $_{GA}\Im_{AL}(R)$
  - Groups tuples of R by grouping attributes, GA, and then applies aggregate function list, AL, to define a new relation.
  - AL contains one or more (<aggregate_function>, <attribute>) pairs.
  - Resulting relation contains the grouping attributes, GA, along with results of each of the aggregate functions.

Example – Grouping Operation

- Find the number of staff working in each branch and the sum of their salaries.

ρ_R(branchNo, myCount, mySum)

branchNo $\Im$ COUNT staffNo, SUM salary (Staff)



branchNo	myCount	mySum
B003	3	54000
B005	2	39000
B007	1	9000

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

Relational Algebra and SQL 1

select Fname as Cheese

from Employee

where Bdate = GETDATE();

Equivalent relational algebra expression:

→ ρCheese(πFname(σBdate=GETDATE()(Employee)))

Relational Algebra and SQL 2

The aggregate function operation.

a. $\rho_{R(\text{Dno, No\_of\_employees, Average\_sal})}(_{\text{Dno}}\Im_{\text{COUNT Ssn, AVERAGE Salary}}(\text{EMPLOYEE}))$.

b. $_{\text{Dno}}\Im_{\text{COUNT Ssn, AVERAGE Salary}}(\text{EMPLOYEE})$.

c. $\Im_{\text{COUNT Ssn, AVERAGE Salary}}(\text{EMPLOYEE})$.

a. select Dno, count(Ssn) as No_of_employees,
avg(Salary) as Average_sal from Employee group by Dno;

b. select Dno, count(Ssn), avg(Salary) from Employee
group by Dno;

c. select count(Ssn), avg(Salary) from Employee;

R

(a)

Dno	No_of_employees	Average_sal
5	4	33250
4	3	31000
1	1	55000

(b)

Dno	Count_ssn	Average_salary
5	4	33250
4	3	31000
1	1	55000

(c)

Count_ssn	Average_salary
8	35125

Order of Operations in Relational Algebra

- Order of operations is procedural, from inside the parentheses to outside
- In the relational model, all relations are sets and eliminate duplicates at each step.

$\sigma_1(\rho_2(\sigma_3(\pi_4(R))))$

- **Assignment**

$T \leftarrow R$

$T_4 \leftarrow \pi_4(R)$
$T_3 \leftarrow \sigma_3(T_4)$
$T_2 \leftarrow \rho_2(T_3)$
$T_1 \leftarrow \sigma_1(T_2)$

The result of evaluating relational expression R is renamed as T and may be used in further expressions

Example - Cartesian product

- List the names and comments of all clients who have viewed a property for rent.

($\Pi_{clientNo, fName, lName}$(Client)) X ($\Pi_{clientNo, propertyNo, comment}$ (Viewing))

client.clientNo	fName	lName	Viewing.clientNo	propertyNo	comment
CR76	John	Kay	CR56	PA14	too small
CR76	John	Kay	CR76	PG4	too remote
CR76	John	Kay	CR56	PG4	
CR76	John	Kay	CR62	PA14	no dining room
CR76	John	Kay	CR56	PG36	
CR56	Aline	Stewart	CR56	PA14	too small
CR56	Aline	Stewart	CR76	PG4	too remote
CR56	Aline	Stewart	CR56	PG4	
CR56	Aline	Stewart	CR62	PA14	no dining room
CR56	Aline	Stewart	CR56	PG36	
CR74	Mike	Ritchie	CR56	PA14	too small
CR74	Mike	Ritchie	CR76	PG4	too remote
CR74	Mike	Ritchie	CR56	PG4	
CR74	Mike	Ritchie	CR62	PA14	no dining room
CR74	Mike	Ritchie	CR56	PG36	
CR62	Mary	Tregear	CR56	PA14	too small
CR62	Mary	Tregear	CR76	PG4	too remote
CR62	Mary	Tregear	CR56	PG4	
CR62	Mary	Tregear	CR62	PA14	no dining room
CR62	Mary	Tregear	CR56	PG36	

Example - Cartesian product and Selection

- Use selection operation to extract those tuples where Client.clientNo = Viewing.clientNo.

$$\sigma_{\text{Client.clientNo = Viewing.clientNo}}((\Pi_{\text{clientNo, fName, lName}}(\text{Client})) \text{ X } (\Pi_{\text{clientNo, propertyNo, comment}}(\text{Viewing})))$$

client.clientNo	fName	lName	Viewing.clientNo	propertyNo	comment
CR76	John	Kay	CR76	PG4	too remote
CR56	Aline	Stewart	CR56	PA14	too small
CR56	Aline	Stewart	CR56	PG4	
CR56	Aline	Stewart	CR56	PG36	
CR62	Mary	Tregear	CR62	PA14	no dining room

- **Cartesian product and Selection can be reduced to a single operation called a *Join*.**

Join Operations

- Join is a derivative of Cartesian product.
- Equivalent to performing a Selection, using join predicate as selection formula, over Cartesian product of the two operand relations.
- One of the most difficult operations to implement efficiently in an RDBMS and one reason why RDBMSs have intrinsic performance problems.

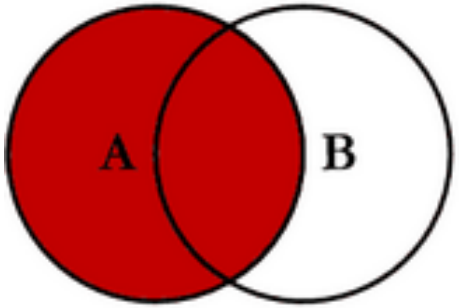
OPERATION	PURPOSE	NOTATION
SELECT	Selects all tuples that satisfy the selection condition from a relation R.	$\sigma_{\text{<selection condition>}}(R)$
PROJECT	Produces a new relation with only some of the attributes of R, and removes duplicate tuples.	$\pi_{\text{<attribute list>}}(R)$
THETA JOIN	Produces all combinations of tuples from R_1 and R_2 that satisfy the join condition.	$R_1 \bowtie_{\text{<join condition>}} R_2$
EQUIJOIN	Produces all the combinations of tuples from R_1 and R_2 that satisfy a join condition with only equality comparisons.	$R_1 \bowtie_{\text{<join condition>}} R_2$, OR $R_1 \bowtie_{\text{(<join attributes 1>), (<join attributes 2>)}} R_2$
NATURAL JOIN	Same as EQUIJOIN except that the join attributes of R_2 are not included in the resulting relation; if the join attributes have the same names, they do not have to be specified at all.	$R_1 *_{\text{<join condition>}} R_2$, OR $R_1 *_{\text{(<join attributes 1>), (<join attributes 2>)}} R_2$ OR $R_1 * R_2$

Reference

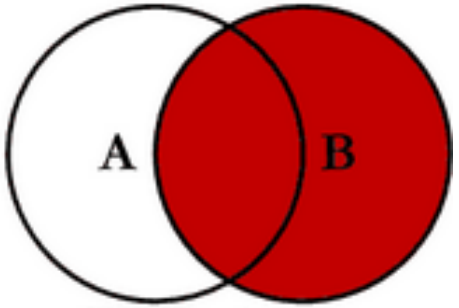
OPERATION	PURPOSE	NOTATION
UNION	Produces a relation that includes all the tuples in R_1 or R_2 or both R_1 and R_2; R_1 and R_2 must be union compatible.	$R_1 \cup R_2$
INTERSECTION	Produces a relation that includes all the tuples in both R_1 and R_2; R_1 and R_2 must be union compatible.	$R_1 \cap R_2$
DIFFERENCE	Produces a relation that includes all the tuples in R_1 that are not in R_2; R_1 and R_2 must be union compatible.	$R_1 - R_2$
CARTESIAN PRODUCT	Produces a relation that has the attributes of R_1 and R_2 and includes as tuples all possible combinations of tuples from R_1 and R_2.	$R_1 \times R_2$
DIVISION	Produces a relation $R(X)$ that includes all tuples $t[X]$ in $R_1(Z)$ that appear in R_1 in combination with every tuple from $R_2(Y)$, where $Z = X \cup Y$.	$R_1(Z) \div R_2(Y)$

Reference

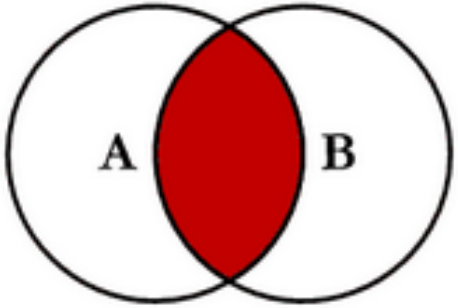
SQL JOINS



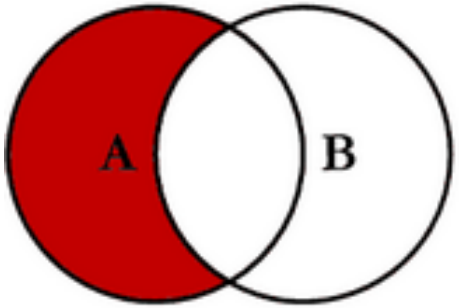

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



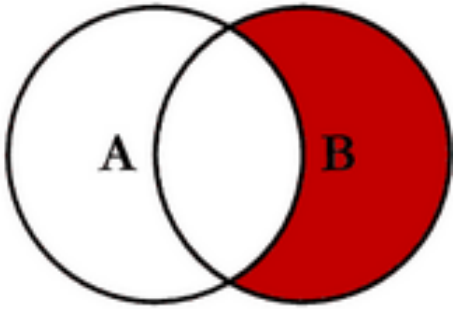

```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



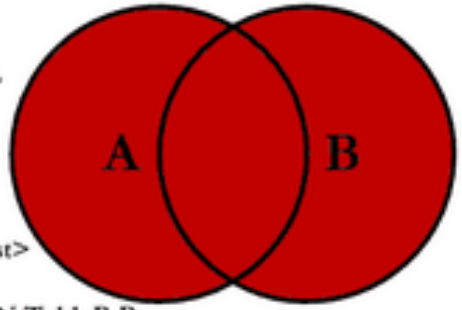

```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



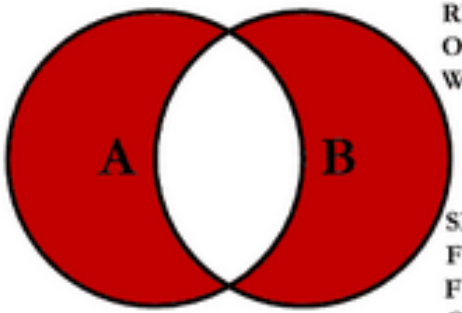

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```

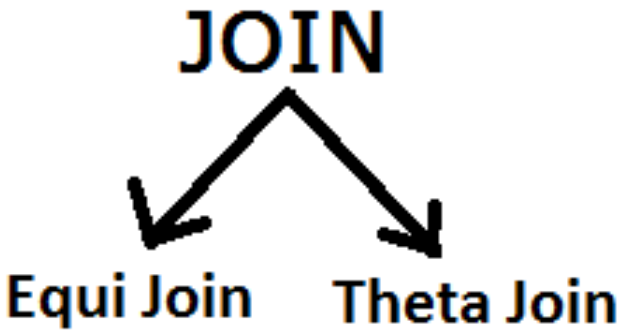



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```


Reference

Join Operations

- Various forms of join operation
  - Theta join
  - Equijoin (a particular type of Theta join)
  - Natural join
  - Outer join
  - Semijoin



Theta join (θ-join)

- **$R \bowtie_F S$**
  - Defines a relation that contains tuples satisfying the predicate **F** from the Cartesian product of R and S.
  - The predicate F is of the form $R.a_i \ \theta \ S.b_i$ where θ may be one of the comparison operators ($<, \leq, >, \geq, =, \neq$).

Theta join (θ-join)

- Can rewrite Theta join using basic Selection and Cartesian product operations.

$$R \bowtie_F S = \sigma_F(R \times S)$$

Degree of a Theta join is sum of degrees of the operand relations R and S. If predicate F contains only equality (=), the term *Equijoin* is used.

Example - Join

student_academic(slno, class, total)

slno	class	total
208501	S5R	211
208502	S5R	137
208503	S5R	210

student_personal(sno, sname, city)

sno	sname	city
208501	ANIL	EKM
208502	BABU	TVM

student_academic $\bowtie_{slno=sno}$ student_personal

slno	class	total	sno	sname	city
208501	S5R	211	208501	ANIL	EKM
208502	S5R	137	208502	BABU	TVM

Example - Equijoin

- List the names and comments of all clients who have viewed a property for rent.

$$(\Pi_{\text{clientNo, fName, lName}}(\text{Client})) \bowtie \text{Client.clientNo} = \text{Viewing.clientNo} (\Pi_{\text{clientNo, propertyNo, comment}}(\text{Viewing}))$$

client.clientNo	fName	lName	Viewing.clientNo	propertyNo	comment
CR76	John	Kay	CR76	PG4	too remote
CR56	Aline	Stewart	CR56	PA14	too small
CR56	Aline	Stewart	CR56	PG4	
CR56	Aline	Stewart	CR56	PG36	
CR62	Mary	Tregear	CR62	PA14	no dining room

Natural join

- **R ⋈ S**
  - An Equijoin of the two relations R and S over all common attributes *x*. One occurrence of each common attribute is eliminated from the result.

Example – Natural Join

student_academic(slno, class, total)

slno	class	total
208501	S5R	211
208502	S5R	137
208503	S5R	210

student_personal(sno, sname, city)

sno	sname	city
208501	ANIL	EKM
208502	BABU	TVM

student_academic ⋈ student_personal

student_academic $\bowtie_{sno=sno}$ student_personal

sno	class	total	sname	city
208501	S5R	211	ANIL	EKM
208502	S5R	137	BABU	TVM

Example - Natural join

- List the names and comments of all clients who have viewed a property for rent.

$(\Pi_{\text{clientNo, fName, lName}}(\text{Client})) \bowtie (\Pi_{\text{clientNo, propertyNo, comment}}(\text{Viewing}))$

clientNo	fName	lName	propertyNo	comment
CR76	John	Kay	PG4	too remote
CR56	Aline	Stewart	PA14	too small
CR56	Aline	Stewart	PG4	
CR56	Aline	Stewart	PG36	
CR62	Mary	Tregear	PA14	no dining room

Outer join

- To display rows in the result that do not have matching values in the join column, use Outer join.
- In natural join, tuples without matching (related) tuples and null tuples are eliminated from Join result. They are considered in outer join.

- **R ⟕ S**
  - (Left) outer join is join in which tuples from R that do not have matching values in common columns of S are also included in result relation.

Example - Left Outer join

- Produce a status report on property viewings.

$\Pi_{propertyNo,\ street,\ city}$(**PropertyForRent**) ⟕ **Viewing**

```
SELECT p.propertyNo, p.street, p.city, v.*
FROM PropertyForRent p LEFT JOIN Viewing v
ON p.propertyNo=v.propertyNo;
```

propertyNo	street	city	clientNo	viewDate	comment
PA14	16 Holhead	Aberdeen	CR56	24-May-01	too small
PA14	16 Holhead	Aberdeen	CR62	14-May-01	no dining room
PL94	6 Argyll St	London	null	null	null
PG4	6 Lawrence St	Glasgow	CR76	20-Apr-01	too remote
PG4	6 Lawrence St	Glasgow	CR56	26-May-01	
PG36	2 Manor Rd	Glasgow	CR56	28-Apr-01	
PG21	18 Dale Rd	Glasgow	null	null	null
PG16	5 Novar Dr	Glasgow	null	null	null

Relational Algebra Operations

T

A	B
a	1
b	2

U

B	C
1	x
1	y
3	z

$T \bowtie U$

A	B	C
a	1	x
a	1	y

(g) Natural join

$T \triangleright_B U$

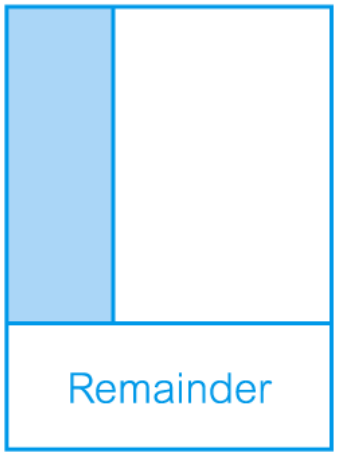
A	B
a	1

(h) Semijoin

$T \rtimes_C U$

A	B	C
a	1	x
a	1	y
b	2	

(i) Left Outer join



(j) Divis on (shaded area)

V

A	B
a	1
a	2
b	1
b	2
c	1

W

B
1
2

$V \div W$

A
a
b

Example of division

Optional

Mathematical Definition of Relation

- Consider three sets D_1, D_2, D_3 with Cartesian Product $D_1 \times D_2 \times D_3$; e.g.

$D_1 = \{1, 3\}$ $D_2 = \{2, 4\}$ $D_3 = \{5, 6\}$

$D_1 \times D_2 \times D_3 = \{(1,2,5), (1,2,6), (1,4,5), (1,4,6), (3,2,5), (3,2,6), (3,4,5), (3,4,6)\}$

  - All combinations of (D_1 , D_2 , D_3)

- Any subset of these ordered triples is a relation.

Optional

Division

- R ÷ S
  - Defines a relation over the attributes C that consists of set of tuples from R that match combination of *every* tuple in S.

Optional

Example - Division

- Identify all clients who have viewed all properties with three rooms.

$(\Pi_{clientNo,\ propertyNo}(Viewing)) \div (\Pi_{propertyNo}(\sigma_{rooms\ =\ 3}(PropertyForRent)))$

$\Pi_{clientNo,propertyNo}(Viewing)$

clientNo	propertyNo
CR56	PA14
CR76	PG4
CR56	PG4
CR62	PA14
CR56	PG36

$\div$

$\Pi_{propertyNo}(\sigma_{rooms=3}(PropertyForRent))$

propertyNo
PG4
PG36

RESULT

clientNo
CR56

Optional

Tuple Relational Calculus

- Interested in finding tuples for which a predicate is true. Based on use of tuple variables.
- Tuple variable is a variable that 'ranges over' a named relation: i.e., variable whose only permitted values are tuples of the relation.
- Specify range of a tuple variable *S* as the Staff relation as:

  Staff(S)

Optional

Tuple Relational Calculus - Example

- To find details of all staff earning more than 10,000:

  $\{S \mid Staff(S) \wedge S.salary > 10000\}$

- To find a particular attribute, such as salary, write:

  $\{S.salary \mid Staff(S) \wedge S.salary > 10000\}$

- SQL:

```
SELECT S.salary from Staff S
where S.salary > 10000;
```

Optional

Example - Tuple Relational Calculus 1

- List the names of all managers who earn more than 25,000.

  $\{S.fName, S.lName \mid Staff(S) \wedge$
  $S.position = \text{‘Manager’} \wedge S.salary > 25000\}$

- SQL:

```
select S.fName, S.lName from Staff S
where S.position = 'Manager'
and S.salary > 25000;
```

Optional

Example - Tuple Relational Calculus 2

- List the staff name who manage properties for rent in Glasgow.

  $\{S.fName, S.lName \mid Staff(S) \wedge (\exists P)$
  $(PropertyForRent(P) \wedge (P.staffNo = S.staffNo) \wedge$
  $P.city = \text{‘Glasgow’})\}$

- SQL:

```
select S.name from Staff S, PropertyForRent P
where P.staffNo = S.staffNo
and P.city = 'Glasgow';
```

Optional

Tuple Relational Calculus

- Can use two *quantifiers* to tell how many instances the predicate applies to:
  - Existential quantifier ∃ (‘there exists’)
  - Universal quantifier ∀ (‘for all’)

- Tuple variables qualified by ∀ or ∃ are called *bound* variables, otherwise called *free* variables.

Optional

Semijoin

- $\mathbf{R} \triangleright_{\mathbf{F}} \mathbf{S}$
  - Defines a relation that contains the tuples of R that participate in the join of R with S.

- Can rewrite Semijoin using Projection and Join:

$$\mathbf{R} \triangleright_{\mathbf{F}} \mathbf{S} = \Pi_{\mathbf{A}}(\mathbf{R} \bowtie_{\mathbf{F}} \mathbf{S})$$

Where **A** is the set of all attributes for **R**

Optional

Example – Semijoin

- List complete details of all staff who work at the branch in Glasgow.

Staff $\triangleright_{\text{Staff.branchNo=Branch.branchNo}}(\sigma_{\text{city='Glasgow'}}(\textbf{Branch}))$

➔ $\Pi_A(\textbf{Staff} \bowtie_F \textbf{Branch})$, where A is the set of all Staff fields

Select * from Staff S where EXISTS (select 1 from Branch B where B.city='Glasgow' and S.branchNo=B.branchNo);

staffNo	fName	lName	position	sex	DOB	salary	branchN
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24- Mar-58	18000	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003

Optional

Conclusions

- Understand the symbols of relational algebra
- Understand the relational Calculus
- Know how to translate between relational algebra and SQL statement